

OT/ICS Lab — Operator Runbook

Lab Operations Runbook

Day-to-day operating notes for the OT/ICS lab. For the operator running the lab, not for a developer editing it. Commands first, prose second. Background and design rationale live in docs/MASTER.md; this file stays operational. Findings referenced by \$N below are in MASTER.md.

1. Daily start

```
# 1. Start Docker Desktop from the Windows start menu (wait until
the
#   whale icon in the system tray is steady, not animated)
# 2. In WSL:
cd /mnt/c/Users/jimmy/Desktop/Projects/ot-ics-lab
docker compose up -d
# 3. Open in browser: http://localhost:1881
```

What “working” looks like:

- **FUXA UI loads** at <http://localhost:1881> — the tank display appears
- **Tank animates** — the dark-blue fill level rises and falls between about 20% and 80% over time
- **Pump label flips ON / OFF** as the tank fills and drains
- **Alarm label stays OFF** (red ACTIVE only appears under attack, not during normal operation)

If the tank is frozen at one value or shows nothing, see §7 below.

Bookmarks for daily access

Make a Chrome bookmark folder called “OT Lab” with these four entries:

URL	What it is
http://localhost:1881	FUXA HMI (operator screen — the tank display)
http://localhost:3000	Grafana dashboards (historian visualization)
http://localhost:8086	InfluxDB web UI (time-series database)
https://localhost:8443	OpenPLC runtime (Editor connects here; web UI exists at this address but daily operation uses the Editor)

Right-click the folder → **Open all** brings up the entire daily working set in one gesture.

2. Daily stop

```

# Daily – pauses containers; resume is fast
docker compose stop

# Resume after a stop:
docker compose start

# Full teardown – REMOVES containers (loses the compiled PLC
program;
# next start needs the program re-uploaded from the Editor)
docker compose down

# Nuclear – REMOVES containers AND named volumes (loses the Editor
# admin user, the FUXA project, the runtime database). Don't use
this
# unless you're intentionally resetting the lab from scratch.
docker compose down -v

```

What persists across each operation:

State	stop	down	down -v
Compiled PLC program (in container)		× re-upload needed	×
Admin user otlabadmin (named volume)			×
FUXA project / device config (bind mount)			
Suricata logs (bind mount)			

Practical rule: stop = no re-upload needed. down = re-upload needed.

Default daily stop is `docker compose stop` — fast, preserves everything.

3. PLC program edits

Only needed when changing the simulation logic itself. Not part of daily operation.

This same procedure is also what to run after any `docker compose down` — see §2 for why.

1. Start the lab: `docker compose up -d` (per §1)
2. On Windows: open **OpenPLC Editor v4** (start menu or `C:\Users\jimmy\AppData\Local\Programs\open-plc-editor\OpenPLC Editor.exe`)
3. **File → Open Project** → select `C:\Users\jimmy\Desktop\Projects\ot-ics-lab\plc\v4\editor-project\project.json`
4. Edit variables in the Variables table on the right, or code in the center editor pane. **An asterisk on the tab title means unsaved.**
5. **Ctrl+S** to save changes to disk.
6. On the Device Configuration screen:
 - IP Address: localhost
 - Click **Connect**
 - Login modal opens — enter credentials from §4 below
 - Status should show ● Connected
7. Click the **Compile** button (the **download-arrow icon**, NOT the triangle Play button — the Play button has a known bug per Finding §19; the download-arrow does the upload-and-run for us).
8. Wait for the bottom-of-screen indicator: **PLC: RUNNING, Scan Count climbing, 0 overruns.**
9. Close the Editor. The PLC keeps running independently — the Editor is just for authoring.

4. Credentials

Lab-only. Low sensitivity (not production secrets), but treated as “not in the repo” so a public clone doesn’t ship a default password.

Service	Username	Password
Editor → Runtime login (port 8443)	otlabadmin	set on first run (store in your password manager or somewhere outside the repo) — see curl below
FUXA UI (port 1881)	(no login configured — open access)	

The otlabadmin user is persisted in the runtime’s `restapi.db` inside the `otlab_openplc_v4_state` named volume. It survives `docker compose stop` and `docker compose down`. It does NOT survive `docker compose down -v`. If you ever down `-v`, the first up `-d` will leave the runtime with no users; create one via the Editor’s first-run flow (it pops up a “create user” modal automatically when no users exist) or via this command, substituting your chosen password:

```
curl -k -X POST https://localhost:8443/api/create-user \
-H "Content-Type: application/json" \
-d '{"username":"otlabadmin","password":"<your-password-here>","role":"admin"}'
```

5. What’s normal vs not

Log message / symptom	Normal?	Why
FUXA tank animating	☐ healthy	Modbus poll FUXA→v4 working
FUXA frozen on one value	⚠	Modbus poll dropped — see §7
[ERROR] Failed to save updated plugin configuration after Compile	☐ NORMAL	The <code>:ro</code> bind mount on <code>plugins.conf</code> is doing its job — see Finding §20
[ERROR] Failed to set PLC state to RUNNING on cold start	☐ NORMAL until a program is uploaded	No compiled program yet — re-upload via Editor (§3)
Editor says “Runtime Version Mismatch”	⚠ Should NOT happen on v4.0.9	If it does, runtime got bumped — confirm <code>docker compose ps</code> shows <code>openplc-runtime:v4.0.9</code>
Editor “Connection failed” / Play button stays grey	⚠ Known bug (§19)	Tick Compile Only , then click the download-arrow Compile button.
Suricata alert files growing in <code>suricata/logs/</code>	☐ healthy if expected	Confirms IDS is watching traffic. Empty = no attacks AND no false positives.

6. Common questions

Do I need the Editor open all the time? No. The Editor is only for changing the program. Once compiled and RUNNING, the PLC executes independently. Close the Editor and the tank keeps cycling.

What if I exit the Editor without saving? Unsaved Editor changes are lost. The PLC program currently running inside the v4 container is **unaffected** — it runs from a compiled .so file, not from the Editor's in-memory state.

Does the lab survive a Windows restart? Yes. Docker Desktop auto-starts after a reboot (if configured); then `docker compose up -d` brings the lab back. The admin user, FUXA project, and (if the container wasn't removed) the compiled PLC program all persist. After a `docker compose down`, you'll need to re-upload the PLC program via the Editor — see §3.

How do I roll back to v3 if v4 breaks? The v3 image is still on disk (ot-ics-lab/openplc:v3-pinned). Rollback would mean: restore the v3 service block in `docker-compose.yml` from git history (pre-commit 8343b9d), edit FUXA's project DB to point at `openplc:502`, and revert Suricata variables. Substantial; talk to me before doing it. Full migration history is in `docs/decisions/0001-phase3-stack.md` and `MASTER.md` §13–§23.

Why are the Editor's Play button and the Compile button doing the same thing now? They aren't exactly — but the Play button has the upstream bug per Finding §19, so I'm using the Compile button (which has no such bug) for normal upload-and-run. The Compile button takes the compile-only path internally when `Compile Only` is ticked; when unticked, it does compile-and-upload like Play would have. Use `Compile` + leave `Compile Only` unticked for daily Build-and-Run.

7. Quick troubleshooting

FUXA showing stale data (frozen tank, last-known values):

```
docker compose restart fuxa
```

Reconnects FUXA's Modbus client. FUXA's driver has no auto-reconnect (Finding §10) so a stuck connection needs operator action. If this recurs more than once a week, that's an environment issue worth investigating.

v4 not listening on port 502 (FUXA shows nothing, tank doesn't animate):

Check the four gates per Finding §15:

```
# Inside the v4 container – should show 0.0.0.0:01F6 (502 in hex)
docker exec otlab-openplc-v4 sh -c \
  'awk "\$4=="0A\" {print \$2}" /proc/net/tcp | grep -i 01F6'
```

If no output, the most common cause is the PLC isn't in RUN state — re-upload the program via the Editor (§3). If the upload finishes but RUN still fails, see the v4 logs:

```
docker logs otlab-openplc-v4 --tail 50
```

Editor won't connect (Runtime Version Mismatch popup):

This is Finding §19 firing. Confirm the runtime is v4.0.9:

```
docker compose ps openplc_v4
# IMAGE column should show: ghcr.io/autonomy-logic/openplc-
runtime:v4.0.9
```

If the image shows :v4.1.0 or :latest, the compose pin was bypassed — revert docker-compose.yml to the committed state. If the image is correct and the popup still appears, tick **Compile Only** in the Editor's Device Configuration screen and use the **download-arrow Compile button**. This bypasses the buggy version check.

Suricata not alerting on something you expected:

Confirm \$OPENPLC_IP points at v4:

```
grep OPENPLC_IP suricata/suricata.yaml
# Should show: OPENPLC_IP: "[172.18.0.5]"
```

If it shows v3 IPs (172.18.0.2), the post-Stop-6 update didn't land — restore from the committed state. After any suricata.yaml edit:

```
docker compose restart suricata
```

docker compose up fails with "mkdir ... file exists" on a bind mount:

Docker Desktop / WSL2 bind-mount race. First fix: retry.

```
docker compose up -d
```

If it fails the same way a second time, fully recreate:

```
docker compose down
docker compose up -d
```

See MASTER.md §27 for the failure-mode class.

Edited a bind-mounted config but restart didn't apply it:

Docker Desktop on Windows + WSL2 caches bind-mount source paths in the container's mount manifest. docker compose restart reuses the stale manifest. Recover by recreating the container:

```
docker compose up -d --force-recreate <service>
```

Equivalent: docker compose down <service> then docker compose up -d <service>. See MASTER.md §27.

Grafana

Grafana health check returns 404 on the name-based endpoint:

Grafana v13 deprecated /api/datasources/name/{name}/health — it now returns 404 with {"message": "Not found"}. Use the UID-based endpoint instead:

```
# 1. List datasources to find the UID
curl -u "$GRAFANA_ADMIN_USER:$GRAFANA_ADMIN_PASSWORD" \
  http://localhost:3000/api/datasources

# 2. Health check by UID (substitute the uid value from step 1)
curl -u "$GRAFANA_ADMIN_USER:$GRAFANA_ADMIN_PASSWORD" \
  "http://localhost:3000/api/datasources/uid/<uid>/health"
```

A healthy InfluxDB datasource returns `{"status": "OK", "message": "datasource is working. N buckets found"}`. See MASTER.md §28.

When to escalate from this runbook

If a symptom doesn't match anything in §5 / §7, or a command in §1-§4 produces an unexpected error, capture the output (terminal copy is enough — no need for screenshots) and bring it to me. Don't try a fourth fix — per the project's troubleshooting rule, three failed attempts is the escalation point.

For the deeper “why” behind any finding referenced here, the canonical location is docs/MASTER.md Findings §1-§23. For Phase 3 architectural choices specifically, docs/decisions/0001-phase3-stack.md.